Neurolabs iOS SDK Integration Guide

Technical Integration Specification

Document ID: NLB-IOS-INTEGRATION-GUIDE
Version: v1.7.21
Date: 2026-10-16
Status: Partner integration reference
Owner: Neurolabs

1. Scope

This guide is for partner iOS apps integrating ‘NeurolabsSDK‘ through Swift Package Manager.

2. Release History

No public API contract has been broken since v1.1.9. New options have been added with backwards-compatible defaults. Recommended defaults: strict shelf guidance with overlays disabled, ‘liveQualityEnabled: true‘, ‘autoCloseAfterCapture: true‘ for single-capture parity flows. ‘NLCameraConfiguration.captureRange(...)‘ is the preferred helper for custom-camera flows that need a minimum capture count and a higher maximum cap. ‘showsCapturedCountLabel‘ hides the ‘N captured‘ chip in custom camera and multi-capture flow UIs.

v1.7.21

No initialiser changes. New APIs are additive. One breaking removal: ‘NLShelfDetector‘ and ‘NLCoreMLYoloDetector‘ are gone. The SDK never constructed them; if your app called them directly, use the SDK’s built-in detection instead.

Behaviour changes, with no opt-in:

- An embedded camera (‘NLCustomCameraView‘, ‘NLCustomCameraViewController‘) that leaves the screen (a push, a tab switch, a modal over it) pauses and resumes on return; its staged photos are kept. The session ends on close, Done or ‘closeNativeCaptureUI‘. An embedded controller must pass ‘onClose‘.
- Opening the SDK camera right after a close waits for the dismissal. An open the SDK cannot present reports ‘didEncounterError‘ with the new code ‘CAMERA_PRESENT_FAILED‘, and a camera that cannot start reports ‘CAMERA_UNAVAILABLE‘. Neither is sent to Sentry.
- In strict mode the camera drops a failed photo before your ‘onCapture‘ hears of it. ‘NLNativeCaptureResult.validationResult‘ carries the quality verdict (nil when nothing validated the capture).
- The SDK camera (‘openNativeCaptureUI‘) refuses a photo before the shutter when the queue would refuse it (‘STORAGE_LOW‘, ‘QUEUE_FULL‘); you get ‘didEncounterError‘ with no capture result.
- Image quality reads luma as Rec.601, as on Android: a saturated colour can get a different verdict than on 1.7.20. No threshold changed.

New APIs (section 14):

- Store context: ‘prefetchStoreContext(outletIds:)‘, ‘storeContextStatus(outletId:)‘, ‘setStoreContextIdentity(_:)‘, ‘clearStoreContextCache()‘.

- Visit KPIs: ‘neurolabsSDK(_:didComputeVisitKPIs:sessionId:)‘ (default no-op), ‘setVisitKPIsEnabled(_:)‘ (on by default), ‘setVisitKPIContext(_:forSession:)‘, ‘computeVisitKPIs(for:context:)‘.

v1.7.20

No API changes. It ships the REX/Smollan audit fixes; 1.7.19 is unchanged, so upgrade to 1.7.20 to get them.

Behaviour changes, with no opt-in:

- With Wi-Fi-only uploads on, a transfer iOS finishes in the background never uses cellular data. The SDK hands a photo to iOS only while the app is in the foreground, and on the next foreground restarts a transfer that 1.7.19 handed off in the background and that has sent nothing (section 6).
- ‘retryFailedUploads()‘ leaves a mission refusal that cannot succeed as sent failed (only ‘flushQueue(force: true)‘ re-sends it), and a call within 60 s of the previous one does nothing (section 6).
- The SDK’s default camera tells the rep why it refused a photo for low storage or a full queue; the host signal is unchanged (section 6).
- "Preparing detector..." holds the shutter for 3 s at most.
- SDK error reports to Sentry never carry a server response body, only the HTTP status and the server’s ‘error_code‘ (section 14).

v1.7.19

No initialiser changes. One additive static method and one stored property set after ‘init‘, both additive for the ABI.

Behaviour changes, with no opt-in:

- A queued photo’s bytes go up through an iOS background ‘URLSession‘, so they keep uploading after the app is suspended or terminated by iOS, and while the phone is locked (not after a user force-quit; see section 6). Forward the app-delegate wake to the SDK to finish sooner (section 6, "Uploads after the app is backgrounded").
- "Review shelf" offers "Retake bay N" on a broken bay link, and an AR multi-bay walk that folds back across a broken link is no longer double-counted (a walk without AR positions makes no attempt).
- The offline recognition build expires after the org’s configured number of days (7 by default) instead of a fixed 7 days.
- The SDK’s process-global Sentry client (started by the Cordova plugin) reports only the SDK’s own errors and crashes, and never replaces a Sentry client your app started first (section 14).

New APIs:

- ‘NeurolabsSDKCore.handleEventsForBackgroundURLSession(_:completionHandler:)‘, optional (section 6).
- ‘NLCameraConfiguration.analyticsEnabled‘ (‘Bool?‘, ‘nil‘ by default: follow the core), so a camera-only app can opt in to camera analytics (section 7, "Camera analytics setting").

v1.7.18

No initialiser changes. The new options and result fields are stored properties and are additive for the ABI.

Behaviour changes, with no opt-in:

- After each photo uploads, its capture metadata (capture contract v1) goes up from a retrying outbox; location and depth only when you opt in (section 6).
- An upload in flight when the app is backgrounded gets iOS's background grace period (section 6).
- A multi-bay walk with a broken bay link is deduped per linked segment instead of giving up on the rest of the walk; the submit sends ‘device_dedup‘ v2 when every segment is trusted, and the walk's stitched realogram (section 9).
- With on-device recognition off, the AR overlay tells touching boxes apart by appearance (section 13).
- The camera follows the core's ‘analyticsEnabled‘, and a camera opened before any ‘NeurolabsSDKCore‘ exists sends no analytics (section 7, "Camera analytics").

New APIs:

- ‘NLConfiguration.captureGeolocation‘ and ‘NLConfiguration.captureDepth‘, both off by default (section 6).
- ‘NLNativeCaptureResult.captureMetadata‘ and ‘NLCustomCameraCapture.captureMetadata‘ (‘NLCaptureMetadata‘, section 6).
- ‘NLMultiBayResult.segments‘ (‘[NLDedupSegment]‘) and ‘NLMultiBayResult.realogram‘ (‘NLStitchedRealogram‘), also on ‘NLNativeMultiBayResultSummary‘ for bridge hosts (section 9).
- ‘NLCameraConfiguration.arAppearanceWithoutRecognition‘, on by default, and its bridge key on ‘NLNativeCaptureConfig‘ / ‘NLNativeCaptureOptions‘ (sections 9 and 13).

v1.7.17

No initialiser changes. The new options are stored properties set after ‘init‘ and are additive for the ABI.

Behaviour changes, with no opt-in:

- An offline chip on the default and AR camera, on by default (section 7).
- "Review shelf" pages through a sweep of more than 8 bays and lists the walk's brands and products under "On this shelf" (section 9).
- The bridge (‘NLNativeMultiBayConfig‘, WebView and Cordova) carries a continuous AR sweep's plan of up to 30 bays (section 9).

New APIs:

- ‘NLCameraConfiguration.showsOfflineIndicator‘ (section 7).
- ‘NLCameraConfiguration.arBoxUnmatchedColor‘ (section 13).
- ‘NeurolabsSDKCore.productDetailsSource‘: product names for a camera opened by ‘openNativeCaptureUI‘ (section 13).
- The bridge keys ‘showsOfflineIndicator‘ and ‘arBoxUnmatchedColor‘ on ‘NLNativeCaptureConfig‘ and ‘NLNativeCaptureOptions‘.

v1.7.16

No initialiser changes. New per-camera options are stored properties set after ‘init‘ and default to the old behaviour; the retained-photo settings and ‘prefetchPolicy‘ are process-wide statics.

Behaviour changes, with no opt-in:

- A new capture is refused with ‘STORAGE_LOW‘ when the queue’s directory has under 500 MB free (section 6). There is no switch to turn this off.
- Captures are sized to a 4032 px long edge at JPEG 0.9 when the host sets no ‘maxImageDimension‘ / ‘imageCompressionQuality‘, and the org’s remote ‘sdkCapture‘ values can resize them further for hosts that set nothing (section 7).
- The AR shutter queues ARKit’s 12 MP still instead of the ~1920x1440 video frame (section 13).

New APIs:

- ‘finalReview‘: an end-of-session review on the default camera (section 7).
- Retained photos: ‘NLConfiguration.maxRetainedPhotos‘, ‘retainedPhotoExpiryDays‘, the ‘retainedPhotosHigh‘ delegate methods, ‘STORAGE_LOW‘, ‘NLQueueStatus.retainedCount‘ / ‘retainedBytes‘ (section 6).
- ‘NLDetectorModels.prefetchPolicy‘ and the image-only quality checks while there is no detector (section 7).
- Capture images default to 4032 px / JPEG 0.9, overridable per org by the remote config (section 7).
- AR options: ‘arBoxStyle‘, ‘arBoxHighlightColor‘, ‘arCoverageStyle‘, ‘arCoverageOutlineColor‘, ‘arBoxPersistence‘, ‘arLiveLabels‘, ‘arLiveCount‘, ‘productDetailsSource‘; 12 MP AR stills (section 13); ‘NLMultiBayOptions.maxSupportedContinuousBays‘ (section 9).

v1.3.2

- Mixpanel ingestion routes to the EU region via ‘NLAnalyticsContext.resolveMixpanelEndpoint(fallback: .mixpanelEndpointEU)‘ at SDK construction; the ‘NLMixpanelAnalyticsTracker‘ default itself stays on the standard US endpoint so partners reusing the class for their own US-region projects are unaffected. Endpoint can be overridden via the ‘NLMixpanelEndpoint‘ Info.plist key or the ‘NL_MIXPANEL_ENDPOINT‘ environment variable (env wins over plist).
- Mixpanel ‘time‘ field is now serialized as a JSON number (epoch seconds). Previously ‘time‘ was sent as a string, which Mixpanel rejected with ‘{"status":0,"error":"time field must be a number"}‘, silently dropping every event.
- ‘NLCustomCameraView.saveAllCaptures‘ keeps the Done button responsive: the synchronous ‘FileManager.removeItem‘ loop for staged-capture files moved into a ‘Task.detached(priority: .utility)‘ block, and the Done label swaps to an inline ‘ProgressView‘ while ‘isPersistingCaptures == true‘. Opacity remains at 1 during persist so the indicator stays visible.
- New ‘NLOperationsCatalogClient‘ (in ‘ProductAuditKit‘) for the operations product catalog, with ‘BarcodeLookupIdentity‘ normalising UPC-E/EAN-8/UPC-A/EAN-13 to GTIN-14 end-to-end. ‘NLOperationsCatalogError.invalidBarcode‘ / ‘.invalidSearchQuery‘ / ‘.malformedResponse‘ are thrown rather than crashing the host with ‘precondition‘.

- ‘NLBDemoApp‘ gains an in-app settings sheet on ‘CustomCameraDemoView‘ (gear icon, top-right). Sliders / steppers / toggles for every tunable knob, bindings wired to existing ‘@AppStorage‘ properties so saved values survive across launches.
- Mixpanel event identity hardened: per-install distinct id persisted in ‘UserDefaults‘, ‘$insert_id‘ (UUID4) on every event for server-side dedup, response body surfaced when ‘debugLogging‘ is true.

v1.3.1

- Restrict barcode symbologies to retail formats (EAN-13, EAN-8, UPC-A, UPC-E) so the scanner stops emitting non-product codes during capture.

v1.3.0

- Angle-off warning threshold relaxed to 12 for less aggressive blocking during normal in-store motion.
- Multi-capture preview deletion: persisted thumbnails are removed when the matching capture is dropped.
- AVFoundation preview recovery hardened against backgrounding / interruption.

v1.2.8

- ‘ProductAuditKit‘ test targets added (no public API change).

v1.2.6 / v1.2.7

- Product Audit (onboarding) MVP: serialise ‘AVAssetWriter‘ finalize and ‘OCRSmartRotationProvider‘ work via in-flight ‘Task‘, off-actor keyframe JPEG writes, completedStepIDs reflect steps that actually ran, step transitions serialised via pending-transition queue.

v1.2.5

- New ‘showsSequenceCounterLabel‘ option in the native capture API for toggling the on-screen sequence counter.
- Persistent custom-camera tuning in the demo app (precursor to the v1.3.2 settings sheet) - preferences survive across runs via ‘UserDefaults‘.
- AVFoundation preview recovery improved on capture failure.
- Strict-rejection preview freeze regression fixed.

v1.2.4

- Release build / Swift 6 archive blockers resolved.

v1.2.3

- ‘NLCameraConfiguration.captureRange(...)‘ helper for bounded multi-capture flows.
- 0.5/1 lens switcher built on AVFoundation, default camera mode starts on the ultra-wide lens when available. Tap-to-focus, lens switch fallback covered by unit tests.
- Custom-camera startup, lens switching, and guidance responsiveness improvements; camera preview shown during warmup, capture button gated until ready.
- Default task routing config uses ‘taskUUID‘.
- Recommended custom-camera parity flow remains strict shelf guidance with overlays disabled, ‘liveQualityEnabled: true‘, ‘autoCloseAfterCapture: true‘ with a single required capture.

v1.2.2

- Analytics tracking + opt-out, partner/device context, Mixpanel defaults, Sentry integration with privacy-safe defaults.

v1.2.1

- Initial lens switcher, memory fixes, bottom-bar centering and UI polish.

v1.2.0

- Docs aligned to v1.1.9 requirements baseline; no SDK code change.

v1.1.9 (baseline)

- Init and per-session routing use ‘taskUUID‘.
- Native detector/model warmup enforced in ‘init‘ and guarded in ‘openCamera‘.
- ‘autoCloseAfterCapture: true‘ keeps preview enabled and closes after Save confirmation.

3. Requirements

- iOS 17.0+ (SDK package declares ‘.iOS(.v17)‘)
- Xcode 16.4+ (release toolchain baseline)
- Swift 5.9+ (Swift 6 toolchains supported)

4. Install (SPM)

Add package dependency (mobile-dist or direct distribution manifest) and link ‘NeurolabsSDK‘.

5. SDK Initialization + Warmup

Uploads use the **operations platform** with a **single key**: the ‘apiKey‘ passed to ‘init‘ seeds the operations credential automatically, and every capture routes through the resumable operations mission flow (‘init-resumable chunked PUT complete‘ per image, ‘submit‘ per session) against ‘api.operations.neurolabs.ai‘. No base URL or separate credential call is needed; use ‘setOperationsApiKey(_:baseURL:)‘ only to rotate the key or to point at a non-production host (staging/dev).

```
import NeurolabsSDK

final class CaptureCoordinator {
    let sdk = NeurolabsSDKCore(configuration: .init(
        apiKey: "<OPERATIONS_API_KEY>",
        debugLogging: true,
        taskUUID: "<DEFAULT_TASK_UUID>" // optional IR-task override (never a mission id)
    ))

    func prepare() {
        // SDK starts image pipeline preparation during init.
        // Optional: observe loading progress through delegate.
    }
}
```

6. Queue Management

```
Task {
    await sdk.setAutoSyncEnabled(true)
    await sdk.setWifiOnlyUploadsEnabled(false)
    await sdk.setDeletePhotosOnUploadSuccess(true)
    await sdk.setMaxQueueSize(200)
    await sdk.setUploadRetryCount(5)

    let status = await sdk.getQueueStatus()
    print("pending=\(status.pendingCount) failed=\(status.failedCount)")

    await sdk.flushQueue()
    await sdk.retryFailedUploads()
}
```

‘retryFailedUploads()‘ gives failed photos a fresh cycle, except a mission refusal that cannot succeed as sent (‘wrong_status‘, ‘submission_mismatch‘, ‘mission_not_found‘, ‘invalid_subsection_name‘, ‘invalid_subsection_id‘, or any ‘not_claimed_by_*‘); only ‘flushQueue(force: true)‘ re-sends those. Since 1.7.20 a call within 60 s of the previous one does nothing, so a retry button or timer cannot reset every backoff.

Inspecting the queue at launch

Before you inspect, withdraw or discard queued work at launch, wait for the queue to finish loading the records an earlier launch left on disk:

```
Task {
    await sdk.awaitQueueHydrated() // the whole pass; returns at once once it has run
    for item in await sdk.getQueuedItems() where item.sessionId == abandonedSessionId {
        await sdk.removeQueuedItem(item.id)
    }
}
```

- It returns once the whole pass has finished, also when the load failed (the queue then holds what it could read). After a re-‘configure()‘ it also waits for the replaced instance’s queue to stop, so what it reports is settled.
- A setter that rebuilds the queue (‘setMaxQueueSize(_:)‘, ‘setUploadRetryCount(_:)‘, a new operations key or base URL) starts a new pass; the call waits for the queue that is current when it returns.
- On an instance that ‘configure()‘ replaced it returns at once. Call it on ‘NeurolabsSDKCore.shared‘.

Retained photos and low storage (v1.7.16)

Only pending, uploading and failed items take a ‘maxQueueSize‘ slot. Uploaded photos the queue keeps on the device (keep-photos mode, ‘setDeletePhotosOnUploadSuccess(false)‘, or photos kept until their session’s submit) never fill the queue; two process-wide settings bound them instead:

```
NLConfiguration.maxRetainedPhotos = 500     // default 500, must be > 0
NLConfiguration.retainedPhotoExpiryDays = 14 // default nil = off, must be > 0

let status = await sdk.getQueueStatus()
print("kept=\(status.retainedCount) bytes=\(status.retainedBytes)")
```

- From 80% of ‘maxRetainedPhotos‘ (rounded up) the SDK calls ‘neurolabsSDK(_:retainedPhotosHigh:limit:)‘ on ‘NeurolabsSDKDelegate‘ once per crossing (re-armed when the count drops back below) with an ‘NLRetainedPhotos‘ (‘count‘, ‘bytes‘, ‘oldestCreatedAt‘). The queue-level twin is ‘queueManager(_:retainedPhotosHigh:limit:)‘ on ‘NLQueueManagerDelegate‘. Both have default empty implementations. At the limit the SDK only logs; it never refuses a capture for retained photos.
- ‘retainedPhotoExpiryDays‘ deletes a kept uploaded photo captured at least that many days ago, files included, at the next flush or launch. Photos kept only for a submit never expire.
- A new capture is refused with ‘STORAGE_LOW‘ (retryable) when the queue’s directory has under 500 MB free. It reaches you through ‘didEncounterError‘ and the failed capture result, as ‘QUEUE_FULL‘ does, and it is a client condition (‘NLError.clientConfigurationCodes‘), never reported to the SDK’s Sentry. Since 1.7.20 the SDK’s default camera (‘openNativeCaptureUI‘) also tells the rep why it refused the photo, for ‘STORAGE_LOW‘ and ‘QUEUE_FULL‘.

Uploads after the app is backgrounded

Since 1.7.19 the SDK hands each queued photo’s bytes to iOS: a background ‘URLSession‘ (one per queue partition) sends them to the photo’s Google Cloud Storage upload session, so the transfer continues while the app is suspended, after iOS terminates it (memory pressure, for example), and while the phone is locked. A user force-quit (swiping the app away) makes iOS cancel the app’s background transfers: nothing is lost, and the next launch resumes each photo from the offset GCS confirmed. The rest stays in your app’s process:

| Step | Where it runs | Credential | |---|---|---| | ‘upload/init-resumable‘, opening the GCS session | the app, when it flushes | operations Bearer | | Sending the photo’s bytes | iOS (‘nsurlsessiond‘) | none: the GCS session URL itself | | ‘upload/complete‘, the mission submit | the app, on the wake or the next foreground | operations Bearer |

The OS request carries no operations credential. iOS stores the GCS session URL with the request; it can write that one photo only and expires within 7 days. To send while the phone is locked, the SDK gives iOS a temporary copy of the photo protected ‘completeUntilFirstUserAuthentication‘ (readable once the phone has been unlocked after a restart). The queue’s own copy keeps ‘completeUnlessOpen‘. The temporary copy is deleted as soon as the transfer finishes or fails, and any copy older than 7 days is swept at launch and on every flush.

Forward the app-delegate wake to the SDK (optional, recommended):

```
func application(_ application: UIApplication,
                 handleEventsForBackgroundURLSession identifier: String,
                 completionHandler: @escaping () -> Void) {
    if NeurolabsSDKCore.handleEventsForBackgroundURLSession(identifier, completionHandler: co
mpletionHandler) {
        return
    }
    completionHandler() // not the SDK’s session: your own handling
}
```

Without it the uploads still finish in the background; the SDK learns of them the next time the app runs. ‘upload/complete‘ and the submit need the operations credential, so they run in the wake only when your app configures the SDK at launch (‘NeurolabsSDKCore.configure‘ and the operations key or auth provider in ‘application(_:didFinishLaunchingWithOptions:)‘); otherwise on the next foreground. A photo whose bytes are with iOS counts as uploading in the queue and spends none of its retry attempts. Nothing is lost when the transfer fails: the photo stays queued and resumes from its confirmed offset.

Since 1.7.20 the SDK hands a photo to iOS only while the app is in the foreground; a flush in the background (a wake, the grace period) uploads in-process, because iOS may hold a transfer created in the background for hours. With Wi-Fi-only uploads on, the transfer never uses cellular data, even when iOS finishes it with the app closed.

A flush also still holds an iOS background task (‘beginBackgroundTask‘) until it ends, so the in-app steps get iOS’s grace period (about 30 s) when the rep locks the phone or switches app.

Cordova: plugin 1.7.19 and later forward the wake itself. From plugin 1.7.20 the plugin adds the method at runtime, and only when ‘CDVAppDelegate‘ has none, so your app compiles as it does without the plugin. If your app’s own ‘AppDelegate‘ implements ‘application(_:handleEventsForBackgroundURLSession:completionHandler:)‘, yours wins: forward to ‘NeurolabsSDKCore.handleEventsForBackgroundURLSession‘ as above (in Swift, cordova-ios requires ‘override‘ on it in a ‘CDVAppDelegate‘ subclass). With an older plugin the uploads still finish in the background and complete on the next foreground.

What reaches the backend

What the operations upload carries:

- Per capture: the JPEG (the full frame, sized by ‘maxImageDimension‘ / ‘imageCompressionQuality‘), with its task and bay (‘subsection_name‘).
- Per capture, after the JPEG uploads: its capture metadata (capture contract v1, ‘PUT .../images/{imageId}/capture-metadata‘), for captures the SDK camera took: shutter time, SDK/device/OS, camera mode, photo size, an EXIF subset (no GPS, no serial numbers), device attitude, camera intrinsics and, on the AR camera, its pose. The detector’s boxes only with ‘sendDetectionsMetadata‘. Location only with ‘NLConfiguration.captureGeolocation‘ AND your app’s location permission (the SDK never asks for it); the LiDAR depth map only with ‘NLConfiguration.captureDepth‘. Both default off. It goes up from its own retrying outbox and never holds up the photo or the submit. Your app reads the same record on ‘NLNativeCaptureResult.captureMetadata‘ and ‘NLCustomCameraCapture.captureMetadata‘.
- Per session: the mission submit’s ‘client_metadata.device_dedup‘ summary for a multi-bay session (section 9).

What stays on the device:

- The capture’s detections always reach your app (‘NLCustomCameraCapture.detections‘, the capture result); ‘sendDetectionsMetadata‘ decides only whether they are kept in the queue record (‘getQueuedItems‘) and uploaded with the capture metadata.
- ‘enablePreviewCropping‘ trims only the thumbnail your app and the review show, to the area the detections cover. The uploaded photo is always the full frame, on iOS and Android.

- ‘.nlrec‘ files (‘arSessionRecording‘, experimental) are AR tooling recordings for replay and calibration. They are not a capture output and are never uploaded.

7. Open Custom Camera

```
import UIKit
import NeurolabsSDK

var capture = NLCaptureConfiguration()
capture.confidenceThreshold = 0.25
capture.iouThreshold = 0.45
capture.maxCaptures = 1
capture.guidanceMode = .strict
capture.showDetectionOverlays = false
capture.showCapturedRegions = false

let config = NLCameraConfiguration.captureRange(
    capture,
    minimumCaptures: 1,
    maximumCaptures: 5,
    cameraMode: .default,
    showCapturePreview: true,
    showPreviewInStrictMode: true,
    showsCapturedCountLabel: false,
    enableValidation: true,
    showAlignmentGuidance: true,
    enableARDetections: false,
    liveQualityEnabled: true,
    liveQualityFPS: 6
)

let handlers = NLCustomCameraHandlers(
    onSave: { capture in
        // Custom post-processing hook for each accepted capture
        // return .performDefault -> SDK also enqueues upload
        // return .skipDefault -> the SDK does not queue it; the photo is
        //   yours to process, but capture.imageFilePath is only valid until
        //   this handler returns. Copy the file first (see section 10).
        return .performDefault
    },
    onSaveAll: { captures in
        // Batch hook (multi-capture flow)
        return .performDefault
    }
)

sdk.openCustomCameraUI(
    configuration: config,
    handlers: handlers,
    onClose: {
        print("Camera closed")
    }
)
```

Recommended capture payload notes:

- ‘liveQualityEnabled: true‘ is the key for pill/rotation/warning/error guidance behavior in the custom camera.
- Keep shelf capture mode with strict guidance and overlays disabled for the custom-camera guidance UI path.

- Use ‘NLCameraConfiguration.captureRange(...)‘ to express a minimum capture requirement plus a larger maximum cap.
- Use ‘showsCapturedCountLabel = false‘ to hide the count chip when you do not want ‘N captured‘ shown.
- ‘guidanceMode: .guidance‘ should only be used as a temporary fallback if a partner explicitly wants looser guidance than the parity path.

Offline chip (v1.7.17)

The default and AR camera show a small chip while the upload queue sees no network: "Offline uploads queued" (or "Offline" with nothing queued), and "Waiting for Wi-Fi uploads queued" while Wi-Fi-only uploads hold photos on a cellular network. It hides while the network is unknown and when the host uploads the photos itself (‘uploadsEnabled == false‘). It is informational only and never gates the shutter.

```
config.showsOfflineIndicator = false // default true
```

On the bridge, ‘showsOfflineIndicator‘ (‘Bool?‘) on ‘NLNativeCaptureOptions‘ and ‘NLNativeCaptureConfig‘ resolves options, then config, then ‘true‘.

Capture image size (v1.7.16)

With no ‘maxImageDimension‘ / ‘imageCompressionQuality‘ set by the host, a capture is sized to a 4032 px long edge at JPEG quality 0.9. Each knob resolves on its own: the host value (‘NLCameraConfiguration‘, which ‘NLNativeCaptureOptions‘ then ‘NLNativeCaptureConfig‘ resolve into), then the org’s remote config, then the SDK default.

- The org values are ‘sdkCapture.maxImageDimension‘ and ‘sdkCapture.imageCompressionQuality‘ in ‘GET /v1/recognition-config‘. They apply only inside 1024-4032 px and 0.5-1.0; anything else is ignored (not clamped) and that knob keeps the default. They are read at capture time, so a config fetched after the camera opened applies to the next shot.
- The tightest cap wins, the aspect ratio holds and nothing is upscaled. Inside every cap an image keeps its bytes when the resolved quality is at least 0.9; a lower one (an org’s 0.7, say) re-encodes it at full size. Width and height caps (‘maxImageWidth‘, ‘maxImageHeight‘) stay host-only.
- To ship the camera’s JPEG untouched, set the host values: a ‘maxImageDimension‘ of 0 or less means no cap, and an ‘imageCompressionQuality‘ of 1 or more keeps an in-cap image’s bytes. Host values always win over the org’s remote config.

End-of-session review: finalReview (v1.7.16)

‘finalReview‘ gives the default (non-AR) camera without a multi-bay plan one review at the end of the session instead of a preview per shot:

```
var config = NLCameraConfiguration.captureRange(
    capture, minimumCaptures: 1, maximumCaptures: 10, cameraMode: .default
)
config.finalReview = true // set after init; default false

// SDK-managed / bridge: the same key, options over config, then false
var options = NLNativeCaptureOptions(allowManualFinish: true)
options.finalReview = true
sdk.openNativeCaptureUI(config: nativeConfig, sessionId: sessionId,
                        options: options)
```

- There is no per-shot preview, whatever ‘showCapturePreview‘ says. Done, or reaching the count that ends the session (‘maxCaptures‘, the auto-close target, or the required count when there is no Done button), opens **Review** with every photo.
- Tapping a photo opens it ("Photo 1 of 2"): quality readout, Crop, Delete, Previous / Next and "Back to review". A crop keeps the capture’s ‘captureId‘; a delete reports ‘onCaptureDeleted‘ with source ‘.review‘.
- **Confirm** saves the photos left, in capture order with crops applied, through your ‘onSaveAll‘ / ‘onSave‘ handlers or the upload queue, and the session result is delivered as usual. **Back to camera** keeps the photos and keeps capturing.
- The close button (X) on an opened photo goes back to Review and discards nothing. The close button on Review, or the camera screen’s own, closes the camera: every photo not yet confirmed is discarded (staged files and the recovery journal deleted) and the session ends as cancelled, with nothing queued. Without ‘finalReview‘ those photos would already have been queued at the shutter.
- A host ‘onClose‘ handler (‘NLCustomCameraHandlers.onClose‘) can intercept that close: the SDK discards and closes only when it returns ‘.performDefault‘.
- Nothing from the session is saved or queued before Confirm. The photos stay staged on the device (and survive an app kill) until then.
- The AR camera and multi-bay sessions ignore it and keep their own flows.

Camera-only integration

If your app presents ‘NLCustomCameraViewController‘ without creating a ‘NeurolabsSDKCore‘, register an operations credential for the detector download first. Without it (or a core) the standalone camera has no detector: capture works, but live detection, the shelf guidance cues and the overlay stay off, and the camera says "Detector unavailable. You can still capture." (reason ‘no_credentials‘), not "Detector downloading".

```
NLDetectorModels.setCredentials(apiKey: operationsKey)              // production
// NLDetectorModels.setCredentials(authProvider: provider, baseURL: stagingURL)
```

The call is idempotent (the same key again changes nothing, so a failure’s retry wait still holds), a new key retries at once, and the most recent call wins, this one or a core’s. A base that is not absolute https with a host is refused.

Camera analytics (1.7.18). An SDK camera follows the ‘analyticsEnabled‘ of the ‘NeurolabsSDKCore‘ that opened it, or, for a camera your app builds itself, of the most recently created core. A camera opened before any ‘NeurolabsSDKCore‘ exists sends no analytics (no Mixpanel events, no event-bus ‘sdk_event‘) for its whole lifetime, even if a core is created later. To have camera analytics, create the core first.

Camera analytics setting (1.7.19). ‘NLCameraConfiguration.analyticsEnabled‘ (a ‘Bool?‘, set after ‘init‘) overrides the core for that camera. ‘nil‘, the default, follows the core as described above. ‘true‘ turns the camera’s analytics on: an app that uses the camera without a ‘NeurolabsSDKCore‘ sets it to opt in, and the camera then sends to the SDK’s default Mixpanel tracker (the token resolves from the core’s ‘NLConfiguration.mixpanelToken‘ when a core exists, then ‘NEUROLABS_MIXPANEL_TOKEN‘, then the Info.plist key, then the SDK default). ‘false‘ turns the camera’s analytics off, even when the core’s are on. The camera’s setting always wins over the core’s:

- ‘nil‘ (default): off with no core or under a core with analytics off; on, through the core’s tracker, under a core with analytics on.
- ‘false‘: off in every case.
- ‘true‘: on in every case; through the core’s tracker under a core with analytics on, otherwise through the SDK’s default Mixpanel tracker.

```
var cameraConfiguration = NLCameraConfiguration(capture: NLCaptureConfiguration())
cameraConfiguration.analyticsEnabled = true   // camera-only app: opt in
```

For a signed-in rep’s bearer, follow this lifecycle:

- **At sign-in,** call ‘setCredentials(authProvider:)‘ with one provider for that rep. You can call it again on each camera open, as long as you pass that same instance. A new provider per call refreshes each time, and the new bearer counts as a new key, which clears the retry wait.
- **On account switch,** call it with a new provider.
- **On sign-out,** make the provider’s ‘token()‘ throw. There is no unregister, and a capture session asks the last provider again while the model is not on the device yet.

The download needs about 195 MB free (3.5 times the model: the archive, its unpacked copy and the compiled model are on disk at once during the install). With less it fails with ‘insufficient_storage‘ and is retried at the next capture session. After an install the SDK deletes older model versions. If the installed model will not load, the SDK deletes it and downloads it once more; if that copy fails too, the status is ‘load_failed‘ until the app restarts. A server answer that is not the model (a captive portal’s page, a body of the wrong length) is ‘download_failed‘, retried after a minute, not ‘sha_mismatch‘.

Detector download network policy (v1.7.16)

The SDK downloads the ~56 MB detector by itself (core start, a credential change, ‘setCredentials‘, a capture session opening). To keep that off cellular:

```
NLDetectorModels.prefetchPolicy = .unmeteredOnly // default .anyNetwork
// set it before creating a NeurolabsSDKCore or registering credentials
```

- With ‘.unmeteredOnly‘ those downloads run only on a network that is neither expensive (cellular, a personal hotspot) nor in Low Data Mode (Low Data Mode holds it on Wi-Fi too). A held download starts by itself when such a network appears, or at once when the policy goes back to ‘.anyNetwork‘.
- While it waits, ‘NLDetectorModels.status(for:)‘ is ‘.notDownloaded‘ and ‘NLDetectorModels.statusReason(for:)‘ is ‘waiting_for_unmetered_network‘ (‘NLDetectorPrefetchNetworkPolicy.waitingForUnmeteredNetworkReason‘), not a failure. An explicit ‘NLDetectorModels.prefetch(_:)‘ ignores the policy, and a model already on the device is used on any network.

Quality checks without a detector (option B). While the detector is not ready (downloading, waiting for an unmetered network, no credential, or failed), the camera runs only the checks that need no detections: hold steady / tap to focus, too dark, too bright, glare, backlight, a finger over the lens, perspective skew, rotate the phone and the multi-bay overlap cues. Point at a shelf, framing, shelf-row, price-tag and product-sharpness rules stay off, and strict mode never refuses a photo for too few detections. The tick after the downloaded model swaps in mid-session turns the full set on, with no restart.

8. WebView Bridge Example

The same capture-range options are available through the native bridge models:

```
let config = NLNativeCaptureConfig(
    guidanceMode: .strict,
    showCloseButton: false,
    maxCaptures: 5,
    allowManualFinish: true,
    minCapturesBeforeDone: 1,
    showsCapturedCountLabel: false,
    cameraMode: .default
)

neurolabsSDK.openNativeCaptureUI(config: config, sessionId: sessionId)
```

Use ‘NLNativeCaptureOptions‘ with the same field names when the bridge payload is built from JS or other app code.

9. Multi-Bay Capture (Long Shelves)

Multi-bay capture handles shelves too long for a single photo: the user side-steps along the shelf and captures one overlapping photo per bay, the SDK guides the side-step, dedups the overlap on device, and reports merged product counts across bays in a single session result.

Enable it by setting ‘NLCameraConfiguration.multiBay‘ (nil = feature off, the default):

```
var config = NLCameraConfiguration()
config.multiBay = NLMultiBayOptions(
    minBays: 2,                   // minimum bays before the session can finish (default 1)
    maxBays: 4,                   // maximum bays per session (default 8; init clamps to 8)
    targetOverlapFraction: 0.30,  // fraction of the previous bay kept visible (default 0.30)
    minDetectionConfidence: 0.5,  // confidence floor for merge/count participation (default
0.5)
    onDeviceDedup: true,          // run overlap dedup at session finish (default true)
    generatePreview: true,        // compose the display-only stitched preview (default true)
    previewMaxDimension: 2048     // preview long-edge cap (default 2048, clamped 512...4096)
)

sdk.openCustomCameraUI(configuration: config, handlers: handlers)
```

Long continuous AR sweeps (v1.7.16). A continuous (shutter-free) sweep on the AR camera commits a bay on every auto-capture, so it may plan up to ‘NLMultiBayOptions.maxSupportedContinuousBays‘ (30). ‘init‘ still clamps to ‘maxSupportedBays‘ (8); set ‘maxBays‘ after init. The capture screen clamps it to ‘minBays...30‘ for ‘captureMode == .continuous‘ on the AR camera and to 8 otherwise, and logs a clamp. From v1.7.17 the bridge (‘NLNativeMultiBayConfig‘, used by the WebView bridge and Cordova) resolves ‘maxBays‘ the same way when the camera opens; in v1.7.16 it clamped to 8 on every mode.

```
var options = NLMultiBayOptions(minBays: 2)
options.maxBays = 20 // after init; honoured only for a continuous AR sweep
config.cameraMode = .ar
config.captureMode = .continuous
config.multiBay = options
```

"Review shelf" for long sweeps (v1.7.17). Past 8 bays the AR flow’s review shows the shelf as pages of consecutive bays (9 5 + 4, 30 8 + 8 + 7 + 7), each stitched when the rep turns to it, with Previous / Next and a swipe. Up to 8 bays nothing changes, and ‘NLMultiBayResult.previewImageData‘ is still the one panorama. No API change.

"On this shelf" (v1.7.17). The same review lists the walk’s facings by brand, with each brand’s share of the recognised products, and by SKU. A facing counts as recognised at a match of 0.6 or more. It needs on-device recognition (section 12) and a ‘productDetailsSource‘ for names and brands; without recognition it shows the product total only. No API change.

On the WebView bridge the same options travel as ‘NLNativeCaptureOptions.multiBay‘ (‘NLNativeMultiBayConfig‘): every field is optional and absent fields take the ‘NLMultiBayOptions‘ defaults. The field names are camelCase and identical across iOS, Android, and Cordova.

Reading merged results:

- Native session result: ‘NLCaptureSessionResult.multiBay‘ (‘NLMultiBayResult?‘, nil unless the session ran multi-bay) carries ‘bays: [NLBaySummary]‘, ‘mergedProducts: [NLMergedProduct]‘, ‘countsByLabel: [String: Int]‘, the stitched preview (‘previewImageData‘ / ‘previewImageFileUri‘), and ‘dedupTrusted‘. From v1.7.18 it also carries ‘segments‘ (‘[NLDedupSegment]‘: ‘fromBay‘, ‘toBay‘, ‘trusted‘, ‘countsByLabel‘, one per run of linked bays) and ‘realogram‘.
- ‘countsByLabel‘ (‘counts_by_label‘ on the wire) is keyed by detector class (‘sku_single‘, ‘sku_multipack‘, price tag, promo, poster), not by product or SKU. Shelf, row and slot positions are in ‘NLMultiBayResult.realogram‘ (‘NLStitchedRealogram‘).
- Use ‘NLMultiBayResult.productCount‘ for a product tally - it sums only ‘sku_single‘ + ‘sku_multipack‘; ‘countsByLabel‘ also carries price/promo/poster buckets, so summing every label over-counts products.

- Bridge / SDK-managed flows: the ‘native_capture_result‘ payload carries ‘multiBay: NLNativeMultiBayResultSummary‘ with ‘countsByLabel‘, ‘baysCount‘, ‘dedupTrusted‘, and ‘previewImageFileUri‘. Preview bytes are intentionally not sent over the bridge; the file URI is present only when a stable spilled file exists. It also carries the optional ‘realogram‘ (‘NLStitchedRealogram‘ with camelCase keys: ‘version‘, ‘coordinateSpace‘, ‘sdkVersion‘, ‘bays‘, ‘shelves[{shelfIndex, rows[{rowIndex, slots[{slotIndex, xMin, yMin, xMax, yMax, bayIndex, detectorClass, trackId, productId?, stackIndex?, score?}]}]}]‘, ‘truncated‘, ‘omittedProductCount‘) and ‘segments‘ (‘[{fromBay, toBay, trusted, countsByLabel}]‘). Each is absent when the SDK built none.
- On the bridge, ‘arAppearanceWithoutRecognition‘ (‘Bool?‘) on ‘NLNativeCaptureOptions‘ and ‘NLNativeCaptureConfig‘ resolves options, then config, then ‘true‘, into ‘NLCameraConfiguration.arAppearanceWithoutRecognition‘.

Meaning of ‘dedupTrusted‘:

- ‘true‘ - counts and ‘mergedProducts‘ are on-device deduped: each physical product instance is counted once across overlapping bays.
- ‘false‘ - an adjacent-pair alignment broke, the dedup timed out, or ‘onDeviceDedup‘ was disabled; counts are then the per-bay sum (an upper bound), not a deduped count.

Upload semantics:

- Each bay photo uploads as a normal capture through the operations queue (resumable mission flow, one image per capture).
- On ‘stopSession‘ the mission submit attaches a compact merge summary (‘{bays, counts_by_label, dedup_trusted, sdk_version, ref_homographies}‘) as structured ‘client_metadata.device_dedup‘ plus, transitionally, a ‘notes‘ duplicate for older backends. The backend stores the summary and serves ‘counts_by_label‘ only when ‘dedup_trusted‘ is true. It does not re-dedup from ‘ref_homographies‘ today.
- SDK-managed flows (‘openNativeCaptureUI‘, ‘openCustomCameraUI‘, bridge-driven capture) record that summary automatically. Only hosts presenting their own custom capture UI must pass ‘NLMultiBayResult.submitNotesJSON()‘ to ‘setMultiBaySubmitNotes(_:forSession:)‘ before ‘stopSession(_:)‘.

Config-only mode:

- There is no partner-facing in-camera mode toggle. The Single Multi-bay capture-mode chip and sheet are internal demo/debug chrome gated behind ‘NLSDKDebug.internalCameraToolsEnabled‘ (default ‘false‘; never enable it in partner builds). Partners control the mode purely via configuration: ‘multiBay‘ set = multi-bay session, ‘multiBay = nil‘ = single-bay session.
- The Neurolabs demo app enables that chrome only in local Debug builds; TestFlight/Release demo builds default to single-bay exactly like partner builds. The multi-bay feature itself is fully available in all builds via configuration - only the demo defaults changed.

10. Post-Processing Hooks

- ‘onSave‘ receives each approved capture.
- ‘onSaveAll‘ receives final batch in multi-capture mode.
- ‘onUploadRequested‘ exists but default queueing is done in ‘onSave‘ / ‘onSaveAll‘ path.

```
let handlers = NLCustomCameraHandlers(
    onSave: { capture in
        MyPostProcessor.shared.enqueue(capture)
        return .skipDefault
    }
)
```

Staged file lifetime

‘.skipDefault‘ stops the SDK queueing the photo. It does **not** hand the file over: ‘imageFilePath‘ / ‘previewImageFilePath‘ point at the camera’s staging folder, and the SDK deletes them once your handler is done with them.

| Handler | The files are valid | The SDK deletes them | |---|---|---| | ‘onSave‘ / ‘onSaveAll‘ (sync) | until the handler returns | after it returns: right away for Save All, when the camera closes for a single Save | | ‘onSaveAsync‘ / ‘onSaveAllAsync‘ | until the handler completes | after it completes, and never while it runs, even after the 30 s save watchdog unblocks the camera and the rep closes it |

- Copy or move the files you keep **before your handler returns**. In an async handler, finish reading them before you return; work you hand to another task after returning must use its own copy.
- An async handler that never completes keeps its files until a later camera’s cleanup finds them older than 24 hours.
- After the watchdog the photos stay in the review, so the rep can Save again. Your late result is ignored (no double save); a retry may call your handler a second time with the same files.
- Android differs: an embedded camera’s files belong to the host until it acknowledges them (PARITY-DECISIONS "Capture file lifetime").

Photo deleted (experimental)

‘onSave‘ / ‘onSaveAll‘ only ever see the final set. To hear about each photo the rep deletes, set ‘NLCameraConfiguration.onCaptureDeleted‘:

```
var config = NLCameraConfiguration()
config.onCaptureDeleted = { deletion in
    // deletion.captureId      the same id onCapture / onSave saw
    // deletion.remainingCount captures still in the session, pending + accepted
    // deletion.source         .review | .captureStrip | .bayPeek
    MyVisitLog.shared.photoDeleted(deletion.captureId, remaining: deletion.remainingCount)
}
```

- Fires exactly once per delete the rep makes, on the main actor, after the capture has left the session and its files are gone.
- ‘source‘ is ‘.review‘ for the review sheet and the final review grid, and ‘.captureStrip‘ for the per-tile delete on the capture strip. ‘.bayPeek‘ is reserved for the AR per-bay peek, which has no delete on iOS today.
- It does not fire for the SDK’s own discards: a retake, a strict-quality reject, a capture your ‘onCapture‘ took with ‘.skipDefault‘, the camera closing, or the upload queue removing a photo (including delete-on-upload-success).
- Nil, the default, sends nothing. It is a stored property, not an ‘init‘ parameter, so it is additive and ABI-safe. The Android SDK has the same contract under the same names.

11. Delegate/Event Callbacks

```
final class SDKDelegate: NeurolabsSDKDelegate {
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didEncounterError error: NLError, sessionId: S
tring?, messageId: String?) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didChangeQueueStatus status: NLQueueStatus) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didUpdateLoadingProgress progress: NLLoadingPr
ogress) {}
}

sdk.delegate = SDKDelegate()
```

‘neurolabsSDK(_:didProduceCaptureResult:for:sessionId:)‘ is optional (it has a default empty implementation). Since 1.7.13 ‘openNativeCaptureUI‘ calls it once for every capture it saves, after trying to queue it:

- ‘success‘ is ‘true‘ when the capture was queued, and ‘message‘ is "Capture saved and queued for upload.".
- ‘success‘ is ‘false‘ when it could not be queued. ‘message‘ says why, and the same error also arrives on ‘didEncounterError‘.
- ‘detections‘ are the capture’s detections, whatever ‘sendDetectionsMetadata‘ says; that option decides only whether they are uploaded with the capture metadata (section 6, "What reaches the backend"). ‘captureMetadata‘ is the capture’s recorded metadata. ‘capturedRegions‘ is empty.
- It arrives after ‘didEnqueue‘ for the same capture. A capture the queue already holds is not reported again.

The whole session’s result still arrives once, at the end, on ‘onSendNativeCaptureResult‘. A camera you open yourself with ‘openCustomCameraUI‘ reports through your own handlers, not this callback.

12. On-Device Recognition (v1.7.1)

Per-account, config-gated: your operations API key resolves YOUR org’s recognition config and vector dataset - enabling an account is a backend flip, no app change. Every stage degrades gracefully; capture is never blocked by recognition.

Source consumers (SwiftPM products ‘RecognitionBootstrap‘, ‘RecognitionEngine‘, ‘RecognitionInterface‘):

```
import RecognitionBootstrap
import RecognitionInterface

let bootstrap = NLRecognitionBootstrap(configuration: .init(
    operationsApiKey: "<your operations key>",
    sdkVersion: NLConstants.sdkVersion
))
// Wrap the components at the SDK boundary (see NLRecognitionProviderBox docs):
if let c = await bootstrap.build() {
    sdk.setRecognitionProvider(NLRecognitionProviderBox(
        isReady: c.isReady,
        recognize: { pb, dets, size in
            await c.recognize(pb, dets.map {
                NLProviderDetection(label: $0.label, score: $0.score, bbox: $0.bbox, id: $0.i
d)
            }, size)
        }
    ))
}
// bootstrap.status / onStatusChange  "provider: ready (N items)" etc.
```

Short-lived session tokens. When your bearer is a session token rather than a fixed key, pass an auth provider and a stable identity instead:

```
extension MySessionTokens: NLRecognitionAuthProvider {} // token() + invalidate()

let bootstrap = NLRecognitionBootstrap(configuration: try .init(
    authProvider: sessionTokens,
    offlineIdentity: rep.id, // stable across tokens; never empty
    sdkVersion: NLConstants.sdkVersion
))
```

The bootstrap asks the provider for the bearer on every request. A 401 calls ‘invalidate()‘ and retries once; a second 401 turns recognition off. The offline record is named after a hash of ‘offlineIdentity‘, so a cold start with no token (the provider throws) still builds from the last sync. Use a new bootstrap when the rep changes.

Offline (stores with no signal). Each build that completes online saves the recognition config and the snapshot’s signed manifest for your credential. A later build that cannot reach the Operations API (offline, timeout, DNS, captive portal, 5xx) builds from it for 7 days after the last full sync (‘NLRecognitionBootstrap.offlineCacheTTL‘), if the cached embedder and every shard pass their checks. ‘bootstrap.offlineSnapshotDate‘ is then the sync time, and ‘bootstrap.statusSummary‘ reads ‘provider: ready (offline, N items, snapshot from YYYY-MM-DD)‘. A 401/403 still turns recognition off and deletes the cached build; a "disabled" config stays off. Have the rep open the app online once a day so the 7 days restart. A build that ended unavailable is retried by a later ‘build()‘ (after 60 s, or 1 h for a definitive failure) and, after a network failure, on reconnect; call ‘build()‘ again when ‘onStatusChange‘ reports ‘.ready‘. The cache lives in ‘Application Support/nl-catalog-snapshots‘ (excluded from backup).

The embedder (the pinned DINOv3-ConvNeXt-tiny export, 768-d) is downloaded by the SDK through ‘GET /v1/device-models‘ with your operations key, checked against the sha256 pinned in source, compiled on the device and cached. Do not bundle it: the SDK never scans the app bundle for a model (decision M5). When the model can’t be had (not entitled, offline, a failed download or compile) the build is skipped as ‘embedder unavailable (<reason>)‘. ‘embedderModelURL‘ is the only local override, for air-gapped or bench setups, and it must still be a 768-d ConvNeXt export.

Binary consumers: ‘RecognitionInterface‘ / ‘RecognitionEngine‘ / ‘RecognitionEngineQdrant‘ ship as xcframeworks on neurolabs-mobile-dist from v1.7.1; the bootstrap itself is source-only this release - wire the chain per the ‘NLRecognitionProviderBox‘ documentation.

Tap-to-product-card: captures expose per-detection matches via ‘sdk.recognitionMatches(forCapture:)‘; drop the SDK’s ‘NLDetectionInspectorView(photo:title:detailsSource:)‘ onto your review screen and feed it an ‘NLProductDetailsSource‘ wrapping ProductAuditKit’s ‘NLProductDetailsResolver‘ (one-liner on the source type’s docs). Offline or unresolved products fall back to a UUID + similarity row automatically.

13. AR Camera Options (v1.7.16)

All are stored properties on ‘NLCameraConfiguration‘, set after ‘init‘, with the same keys on ‘NLNativeCaptureConfig‘ and ‘NLNativeCaptureOptions‘ (resolved options, then config, then the default), except ‘productDetailsSource‘ (see below). Every default keeps the earlier look. Wire values decode leniently: "standard" is an alias of "quiet", an unknown value reads as the default and a malformed colour draws the default colour.

```
var config = NLCameraConfiguration()
config.cameraMode = .ar
config.arBoxStyle = .highlight             // .quiet (default), .identity, .highlight
config.arBoxHighlightColor = "#22C55E"     // #RRGGBB, default #22C55E
config.arBoxUnmatchedColor = "#FACC15"     // #RRGGBB, default #FACC15
config.arCoverageStyle = .tintAndOutline   // .tint (default), .outline, .tintAndOutline
config.arCoverageOutlineColor = "#FACC15"  // #RRGGBB, default #FACC15
config.arBoxPersistence = .session         // .standard (default), .session
config.arLiveLabels = true                 // experimental, default false
config.arLiveCount = true                  // experimental, default false
config.productDetailsSource = detailsSource // names for arLiveLabels
```

- ‘arBoxStyle = .highlight‘ draws every world-locked box as a solid outline in ‘arBoxHighlightColor‘, with no fill. ‘NLARBoxStyle‘ is non-frozen: an exhaustive ‘switch‘ over it needs ‘@unknown default‘.
- With ‘arLiveLabels‘ on and a ready recognition provider, a highlight box that is stable, has been looked at, and has no product name latched on it (no match, or every look below 0.6) is drawn in ‘arBoxUnmatchedColor‘ instead. A box not looked at yet stays ‘arBoxHighlightColor‘. With the labels off, every box keeps ‘arBoxHighlightColor‘.
- ‘arCoverageStyle‘ shows the captured shelf as a tint, an outline on the fitted shelf plane in ‘arCoverageOutlineColor‘ around the captured bays, or both. With no plane fit yet every style shows the tint.

- ‘arBoxPersistence = .session‘ keeps every box that settled for the rest of the session as a dimmed outline (at most 300). ‘.standard‘ fades a box once tracking drops it. Presentation only: counts and coverage are unchanged.
- ‘arLiveLabels‘ draws the top-1 product name on each stable box, only at a match score of 0.6 or more. It needs a ready recognition provider (‘setRecognitionProvider(_:)‘, section 12) and ‘productDetailsSource‘ (an ‘NLProductDetailsSource‘, the seam ‘NLDetectionInspectorView‘ takes) to turn catalog UUIDs into names. Without either there are no labels.
- ‘productDetailsSource‘ on ‘NLCameraConfiguration‘ reaches a camera you open with ‘openCustomCameraUI‘ or present as ‘NLCustomCameraViewController(configuration:)‘ (the camera-only path, section 7). ‘openNativeCaptureUI‘ builds its own configuration, so from v1.7.17 a bridge host sets ‘NeurolabsSDKCore.productDetailsSource‘ before opening the camera instead; a source on the configuration wins. It cannot travel in the bridge’s JSON. Before v1.7.17 ‘arLiveLabels‘ showed no names through ‘openNativeCaptureUI‘.
- ‘arLiveCount‘ shows a "~ N products" chip: the distinct stable boxes seen this session. It is an estimate; the review keeps the exact deduplicated count.
- With recognition off, there are no product names or unmatched colour, and a box is told apart from its neighbours by its position, not by which product it is: two identical facings that touch can be harder to keep apart. From v1.7.18 ‘arAppearanceWithoutRecognition‘ (on by default) adds appearance to that: the same ConvNeXt model, downloaded for appearance only for an org Operations grants it to, or else a colour histogram. The default camera never downloads it.

12 MP AR stills. The AR shutter (single, multi-bay, guided and continuous) queues ARKit’s full-resolution still (‘ARSession.captureHighResolutionFrame‘, 12 MP on current iPhones) instead of the ~1920x1440 video frame, while tracking keeps running. It goes through the same image sizing (4032 px / JPEG 0.9 by default, section 7). No API change.

- It falls back to the video frame when there is no running session (‘no_session‘), on an ARKit error (‘capture_error‘), with no still within 1.5 s (‘timeout‘), for a still in the other orientation (‘orientation_mismatch‘), or for a still no larger than the video frame (‘not_larger‘). The reason goes to the device log only.
- Until ARKit returns the still, the shutter shows a spinner and is disabled, so a second tap does nothing and nothing is queued.

14. Notes

Store context for visit KPIs (since 1.7.21)

A visit’s KPIs need the outlet’s store context (targets, assortment, planograms and the org’s metric definitions). The SDK fetches, verifies and caches it per credential for the org’s offline TTL (default 7 days). Outlet ids are Operations outlet UUIDs.

```
let result = await sdk.prefetchStoreContext(outletIds: routeOutletIds)  // never throws; resu
lt.refreshFailure is a code
let status = await sdk.storeContextStatus(outletId: outletId)           // cache only, no net
work
sdk.setStoreContextIdentity(currentUser.id)  // only with setOperationsAuthProvider; hashed,
never stored or sent
await sdk.clearStoreContextCache()            // sign-out
```

‘status.state‘ is ‘.ready‘, ‘.expired‘ or ‘.missing‘ (switch with ‘@unknown default‘), and ‘status.reason‘ is a stable code (‘not_assigned‘ means the outlet id is not your org’s; treat an unknown code as missing). ‘device_kpis‘ are submitted only for a context that is ready and ‘isSignatureVerified‘; the SDK pins the production key ‘store-context-v1‘ and requires signatures. Staging-backed builds verify with the staging key ‘store-context-v2‘, trusted only on a Neurolabs staging operations host; a production base only ever trusts v1. The full code list is in the README ("Store context (visit KPIs)").

Visit KPIs (since 1.7.21)

After an SDK-managed session finishes with at least one capture, the SDK computes the visit’s KPIs on the device and calls ‘neurolabsSDK(_:didComputeVisitKPIs:sessionId:)‘ (default no-op, main actor). On by default; ‘setVisitKPIsEnabled(false)‘ turns it off.

```
sdk.setVisitKPIContext(NLVisitKPIContext(outletId: outletId, sectionsByCaptureId: [:], sectio
nsByBayIndex: [:]),
                       forSession: sessionId)          // before the session finishes; refres
hes the store context
let result = try await sdk.computeVisitKPIs(for: sessionResult, context: context)  // custom
capture UI; read-only
```

‘NLVisitKPIResult‘ carries the canonical ‘nl.visit_kpis‘ v1 JSON (‘payloadJSON‘), ‘isProvisional‘ (always true on the device: label it "Provisional"), ‘storeContext‘ (‘.used(offline:)‘, ‘.missing‘, ‘.expired‘), ‘perfectStoreScoreBp‘, ‘actionCount‘ and ‘willSubmit‘. Without a store context, show gaps and price-tag presence only, as "Needs store targets". The payload rides the mission submit as ‘client_metadata.device_kpis‘ only when the store context was ready and signature-verified and the payload passes the backend’s checks; a ‘400 invalid_device_kpis‘ makes the SDK resubmit once without it. Details are in the README ("Visit KPIs").

Telemetry and privacy (Sentry)

The SDK reports errors to the Neurolabs Sentry project under this policy, the same on iOS, Android and Cordova:

- **Only the SDK’s own errors and crashes.** The SDK reports the errors it captures itself, and crashes whose crashed thread runs SDK code. A crash in your own code, an app hang, a watchdog termination or a failed request of your app is never sent.
- **Your app is never instrumented.** No swizzling, and no network, app-hang, watchdog, breadcrumb or session tracking. The SDK’s errors go through a private Sentry client, so your own Sentry setup is untouched. The one process-global path is ‘NLSentryCrashReporter.bootstrap(...)‘, which installs a crash handler only when your app (or the Cordova plugin) calls it; it applies the same filter. It never replaces a Sentry client your app already started: if one is running, ‘bootstrap‘ starts nothing and the SDK reports its own errors through its private client only. If your app starts Sentry *after* ‘bootstrap‘, yours replaces the SDK’s (Sentry’s global start is last-writer-wins), and the SDK’s own errors still go through its private client.

- **No credentials or personal data.** ‘sendDefaultPii‘ is off. Messages are scrubbed before they are sent: a URL keeps only its scheme, host and path, so a signed URL’s credentials never leave the device. Since 1.7.20 an error that came from a server response carries only the HTTP status and the server’s ‘error_code‘, never the response body.
- **Opt out** with ‘sentryEnabled: false‘ (or ‘analyticsEnabled: false‘) on ‘NLConfiguration‘. Nothing is sent from an XCTest process.

Other notes

- Per-session routing is supported via native capture config task UUID overrides.
- Ensure ‘NSCameraUsageDescription‘ is present in app ‘Info.plist‘.